BRNO UNIVERSITY OF TECHNOLOGY
VYSOKÉ UČENÍ TECHNICKÉ V BRNĚ

FACULTY OF INFORMATION TECHNOLOGY
FAKULTA INFORMAČNÍCH TECHNOLOGIÍ

DEPARTMENT OF COMPUTER SYSTEMS
ÚSTAV POČÍTAČOVÝCH SYSTÉMŮ

3D NEURAL CELLULAR AUTOMATA

3D NEURÁLNÍ CELULÁRNÍ AUTOMATY

BACHELOR'S THESIS
BAKALÁŘSKÁ PRÁCE

AUTHOR **MICHAL REPČÍK**
AUTOR PRÁCE

SUPERVISOR **Ing. KAREL FRITZ,**
VEDOUCÍ PRÁCE

BRNO 2026

Abstract

Do tohoto odstavce bude zapsán výtah (abstrakt) práce v anglickém jazyce.

Abstrakt

Do tohoto odstavce bude zapsán výtah (abstrakt) práce v českém (slovenském) jazyce.

Keywords

Sem budou zapsána jednotlivá klíčová slova v anglickém jazyce, oddělená čárkami.

Klíčová slova

Sem budou zapsána jednotlivá klíčová slova v českém (slovenském) jazyce, oddělená čárkami.

Reference

REPČÍK, Michal. *3D Neural Cellular Automata*. Brno, 2026. Bachelor's thesis. Brno University of Technology, Faculty of Information Technology. Supervisor Ing. Karel Fritz,

3D Neural Cellular Automata

Declaration

I hereby declare that this Bachelor's thesis was prepared as an original work by the author under the supervision of Ing. Karel Fritz I have listed all the literary sources, publications and other sources, which were used during the preparation of this thesis. I have used ChatGPT to correct spelling and other language mistakes. I used Grok when working on the software.

. . . . . .
Michal Repčík
November 25, 2025

Acknowledgements

I would like to express my sincere thanks to my supervisor, Ing. Karel Fritz, for his guidance and support.

Contents

1 Introduction **3**

2 Abstract **4**

3 Theoretical Foundations **5**

3.1 Prerequisites 5

3.2 Cellular Automata 5

3.3 Neural Cellular Automata 7

4 Conclusion **9**

Bibliography **10**

List of Figures

Chapter 1

Introduction

TODO

Chapter 2

Abstract

TODO

Chapter 3

Theoretical Foundations

3.1 Prerequisites

This thesis assumes the reader is familiar with basic concepts of deep learning, in particular convolutional neural networks, gradient-based optimisation, and automatic differentiation in PyTorch [14, 4].

A comprehensive introduction to these topics can be found in Goodfellow et al. [7] and in the official PyTorch documentation [4].

The rest of the thesis is self-contained with respect to cellular automata and their neural extensions.

3.2 Cellular Automata

Cellular automata (CA) are discrete computational systems that consist of a regular grid of cells, each of which can take on a finite number of states. The state of each cell evolves in parallel over discrete time steps according to a local update rule, which depends on the states of its neighboring cells [11].

Formally, a cellular automaton is defined by:

- A **lattice** of cells, typically 1D, 2D, or 3D.
- A **finite set of states** $\mathcal{S}$ that each cell can occupy.
- A **neighborhood function** $\mathcal{N}$ that defines which cells influence a given cell.
- An **update rule** $f : \mathcal{S}^{|\mathcal{N}|} \to \mathcal{S}$ that computes the next state [9].

CA were pioneered by von Neumann in the 1940s for self-replication studies [13], and later popularized by Wolfram's classification [19, 20] into four classes based on behavior:

- **Class 1 - Uniform**: quickly reaches a single state.
- **Class 2 - Periodic**: static or oscillating patterns.
- **Class 3 - Chaotic**: unpredictable, noisy evolution.
- **Class 4 - Complex**: long-lived structures and computation.

3.2.1 One-Dimensional Cellular Automata

The simplest non-trivial cellular automata live on a 1D line of cells. Each cell looks only at itself and its two immediate neighbours (radius = 1, neighbourhood size = 3). With two states (0 or 1), there are exactly 256 possible elementary rules, numbered from 0 to 255 in Wolfram's convention [19].

A rule number is a binary representation of the next-state lookup table. For example:

- Rule 90 produces the Sierpiński triangle from a single live cell (Class 3).
- Rule 30 creates chaotic, triangle-filled patterns that look random and are often used as pseudorandom generators.
- Rule 110 (Class 4) is the smallest known universal 1D CA - it supports gliders and glider collisions that can perform any computation [5].

3.2.2 Two-Dimensional Cellular Automata

In two dimensions the most common neighbourhood is the 3x3 Moore neighbourhood (8 surrounding cells + centre). With two states this gives $2^9 = 512$ possible inputs, so rules are usually described by birth/survival conditions instead of raw numbers.

The most famous example is Conway's Game of Life introduced in 1970 [6]. A cell becomes or stays alive if it has exactly 3 live neighbours (birth) or 2-3 live neighbours (survival); otherwise it dies. Despite these extremely simple rules, Game of Life exhibits:

- still lifes (e.g. block, beehive),
- oscillators (e.g. blinker, toad),
- spaceships (e.g. glider, lightweight spaceship),
- glider guns and other complex constructions.

Game of Life is Turing-complete [1] and has inspired almost all later work on self-organising and growing neural cellular automata, including the 2D Growing CA by Mordvintsev [12] that this thesis builds upon and extends into three dimensions.

3.2.3 Three-Dimensional Cellular Automata

In three dimensions, the Moore neighbourhood includes up to 26 adjacent cells (face-, edge-, and corner-adjacent) in a 3×3×3 cube. With binary states, this creates $2^{27} \approx 134$ million possible neighbourhood configurations, so practical 3D CA typically use totalistic or outer-totalistic rules that depend only on the sum of live neighbours (not their exact positions) [20].

Notable examples include:

- Von Neumann's 29-state self-replicating automaton (1950s), the foundational model for CA self-reproduction [13].
- Brian's Brain (1990s, 3-state rule): creates expanding "fireworks" patterns via refractory periods [10].

- 3D Life variants (e.g. B3/S45678 or B5678/S45678): produce branching, crystal-like growths.

3D CAs have been studied since the 1950s but gained traction in the 2000s with better hardware for simulation [18]. Neural CA build on this by learning compact update rules that scale efficiently on GPUs, enabling stable 3D self-organisation.

3.2.4 Differentiability and Continuous Cellular Automata

This thesis uses a discrete 3D lattice, but the learned update rule is fully differentiable end-to-end. This allows the same gradient-based training that is used in ordinary neural networks [2].

This connects Neural CA to continuous cellular automata, where states are real numbers (not just zeroes and ones) and updates use smooth functions instead of hard rules.

SmoothLife from 2011 is the first clear example [15]. It's a generalization of Conway's Game of Life to a continuous domain, replacing the infinite grid with an "effective grid" averaged over disks, and turning the discrete rules into differential equations with smooth interpolation. The resulting dynamics produce continuously evolving glider-like objects and turbulent backgrounds on a real-valued field instead of binary alive/dead cells.

Lenia, from eight years later, builds on that idea [3]. It generalizes Game of Life to continuous space, time, and states, using parameterized growth kernels and a growth function based on weighted neighbor sums. Systematic exploration of the parameter space revealed hundreds of stable solitary patterns exhibiting translation, oscillation, and shape change — collectively described by the author as an artificial ecology.

These continuous setups showed gradient-based training could handle complex patterns long before neural networks were applied to cellular automata [12, 2]. The Neural CA in this thesis adopt exactly this differentiability while keeping a discrete voxel grid, combining the advantages of precise 3D representation with the training stability of continuous models.

3.3 Neural Cellular Automata

Neural Cellular Automata (NCA) replace hand-crafted update rules with a small learnable neural network, usually a few convolutional layers. The grid remains discrete, but the cell state becomes a real-valued vector, making the whole system differentiable and trainable with gradient descent.

The breakthrough work is "Growing Neural Cellular Automata" by Mordvintsev et al. [12]. They trained a 16-channel 2D CA on semi-complex targets (lizards, emoji, butterflies, etc.) and discovered three natural training regimes:

- **Growing models** are trained only to generate the target pattern from a small seed. As the authors note, "Growing models were trained to generate patterns, but don't know how to persist them. Some patterns explode, some decay, but some happen to be almost stable or even regenerate parts!" [12] (Experiment 1).

- **Persistent models** are explicitly trained to keep the pattern alive for hundreds of steps. Interestingly, "they often develop some regenerative capabilities without being explicitly instructed to do so" [12] (Experiment 2).

- **Regenerating models** experience random damage at every training step (e.g. large parts of the pattern are zeroed out). These models learn extremely strong regeneration, especially in the central region, as "regenerating models were subject to pattern damages during training, so their regenerative capabilities are much stronger, especially in the central area" [12] (Experiment 3).

All three regimes use the same tiny update network (~ 100 parameters) and the same fixed perception filters: identity, Sobel gradients (x/y), and a 3×3 blur. The only difference is the training objective and whether damage is injected.

Follow-up work explored continuous-space variants (Lenia [3]), self-classifying embryos [8], and persistent memory channels [16], but almost everything stays in 2D and uses monolithic training scripts.

Early attempts at 3D NCA exist, such as the voxel-based implementation by Rossi and Orsini [17] (originally released under the GitHub username SkyLionx), which directly extends Mordvintsev's model to 3D using PyTorch and .vox targets. It supports growth from seeds, damage recovery, and 3D visualization, but ties perception/update logic into a single Colab notebook, limiting reusability for extensions like custom kernels or LLM-generated targets.

This thesis builds on such prototypes by introducing a fully modular architecture that separates these components for easier prototyping and research.

Chapter 4

Conclusion

TODO

Bibliography

[1] Berlekamp, E. R.; Conway, J. H. and Guy, R. K. *Winning Ways for Your Mathematical Plays, Volume 2*. Academic Press, 1982.

[2] Béna, G.; Faldor, M.; Goodman, D. F. M. and Cully, A. *A Path to Univrsal Neural Cellular Automata*. 2025. Available at: https://arxiv.org/abs/2505.13058.

[3] Chan, B. W.-C. *Lenia: Biology of Artificial Life*. 2019.

[4] Community, P. *PyTorch Documentation*. 2025. Available at: https://docs.pytorch.org/stable/index.html.

[5] Cook, M. *Universality in Elementary Cellular Automata*. 2004.

[6] Gardner, M. *Mathematical Games: The fantastic combinations of John Conway's new solitaire game "Life"*. 1970. Available at: https://web.stanford.edu/class/sts145/Library/life.pdf.

[7] Goodfellow, I.; Bengio, Y. and Courville, A. *Deep Learning*. MIT Press, 2016. Available at: https://www.deeplearningbook.org.

[8] Horváth, D. and Levin, M. *Self-Classifying MNCA Distill*. 2021. Available at: https://distill.pub/2021/self-classifying-mnca/.

[9] Ilachinski, A. *Cellular Automata: A Discrete Universe*. World Scientific, 2001. ISBN 981-02-4623-4.

[10] Kinderen, E. de and Wuensche, A. 3D CA in a fluid dynamics PDE. In: *Proceedings of the Second International Conference on Evolvable Systems: From Biology to Hardware*. Springer, 1998.

[11] Marinescu, D. C. Chapter 2 - Nature-Inspired Algorithms and Systems. In: Marinescu, D. C., ed. *Complex Systems and Clouds*. Boston: Elsevier, 2017, p. 33–63. Computer Science Reviews and Trends. ISBN 978-0-12-804041-6. Available at: https://www.sciencedirect.com/science/article/pii/B9780128040416000025.

[12] Mordvintsev, A.; Randazzo, E.; Niklasson, E. and Levin, M. *Growing Neural Cellular Automata*. 2020. Available at: https://distill.pub/2020/growing-ca/.

[13] Neumann, J. von. *Theory of Self-Reproducing Automata*. University of Illinois Press, 1966.

[14] Paszke, A.; Gross, S.; Massa, F.; Lerer, A.; Bradbury, J. et al. *PyTorch: An Imperative Style, High-Performance Deep Learning Library*. 2019. Available at: https://arxiv.org/abs/1912.01703.

[15] Rafler, S. *Generalization of Conway's "Game of Life" to a continuous domain – SmoothLife* arXiv preprint arXiv:1111.1567. 2011. Available at: https://doi.org/10.48550/arXiv.1111.1567.

[16] Randazzo, E.; Mordvintsev, A. and Levin, M. *Persistent Memory in Growing Neural Cellular Automata* Google Research Blog. 2021. Available at: https://ai.googleblog.com/2021/12/persistent-memory-in-neural-cellular.html.

[17] Rossi, F. and Orsini, M. *3D Growing Neural Cellular Automaton*. GitHub, 2021. Available at: https://github.com/SkyLionx/3d-cellular-automaton. Originally published under username SkyLionx.

[18] Tobias, P. A. G. *3D Cellular Automata*. 2009. Available at: https://eprints.bournemouth.ac.uk/17408/.

[19] Wolfram, S. *Statistical Mechanics of Cellular Automata*. 1983.

[20] Wolfram, S. *A New Kind of Science*. Wolfram Media, 2002. ISBN 1-57955-008-8. Available at: https://www.wolframscience.com/nks/.